

Procedural Dungeon generator

Pablo jacobs
Game development
Year 2

The Dungeon:

I wanted to make a dungeon generator that is based on [binding of isaac](#) world generator

my first renditions/attempts at making a world generator I tried to look at wave function collapse and feed my generator rules based on the given grid.

I have let go obviously of this attempt because I had a hard time to understand wave function collapse on a writing level (so conceptual I understand it)

Inspiration:

In the binding of isaac the rooms get cross stepped added to each other like the “**Random step**” algorithm. I have also chosen to not let it go diagonal so it will always make a path that is possible to walk from start to end.

The random step I created was a randomizer between a VectorInt array, that once a direction is chosen it will try that direction until placed.

Another thing that binding of isaac does as well, is separating the first room far away from the boss room. I personally liked this concept and wanted to continue using this rule, as it is a good game design rule to make players experience as much as possible.

I analyse the area with written functions for the world like a [GetNeighbours\(\)](#), [GetFurthest\(\)](#) or [MiddleTile\(\)](#) which can give relevant data to build rules upon for my rooms.

The World Rules:

The world is based on a given size and a Given or random seed that will determine the random steps placed within the world.

Other than that I have implemented a parameter on the amount of combat rooms that can spawn and on the build speed the world generates its tiles which is purely visual as the calculations have already been set in stone before it reveals the tiles

Worthy note rules to those parameters are:

1. When build speed is 0 it skips the entire coroutine and enables all the objects immediately
2. Seed size can only be between (-10000, 10000) when randomized
3. Registered Seeds can be 8 digits long
4. World size can only be between 16 - 64 steps
5. Combat amount is only between 0 - 6
(because 6 is the maximum it can handle when the world is at 16 size)
6. Build speed can only be between 0 and 0.1 seconds
7. When Build speed is 0 it skips the entire visual and does it all instantly
(Instead of generating it on update speed)

The Room Rules:

Colors are set to the once in the video, and the order of these rules are also how it is executed within the generator.

Rule 1: The Exit tile is the furthest tile from the first placed tile

Rule 2: The Start is the furthest tile from the Exit tile

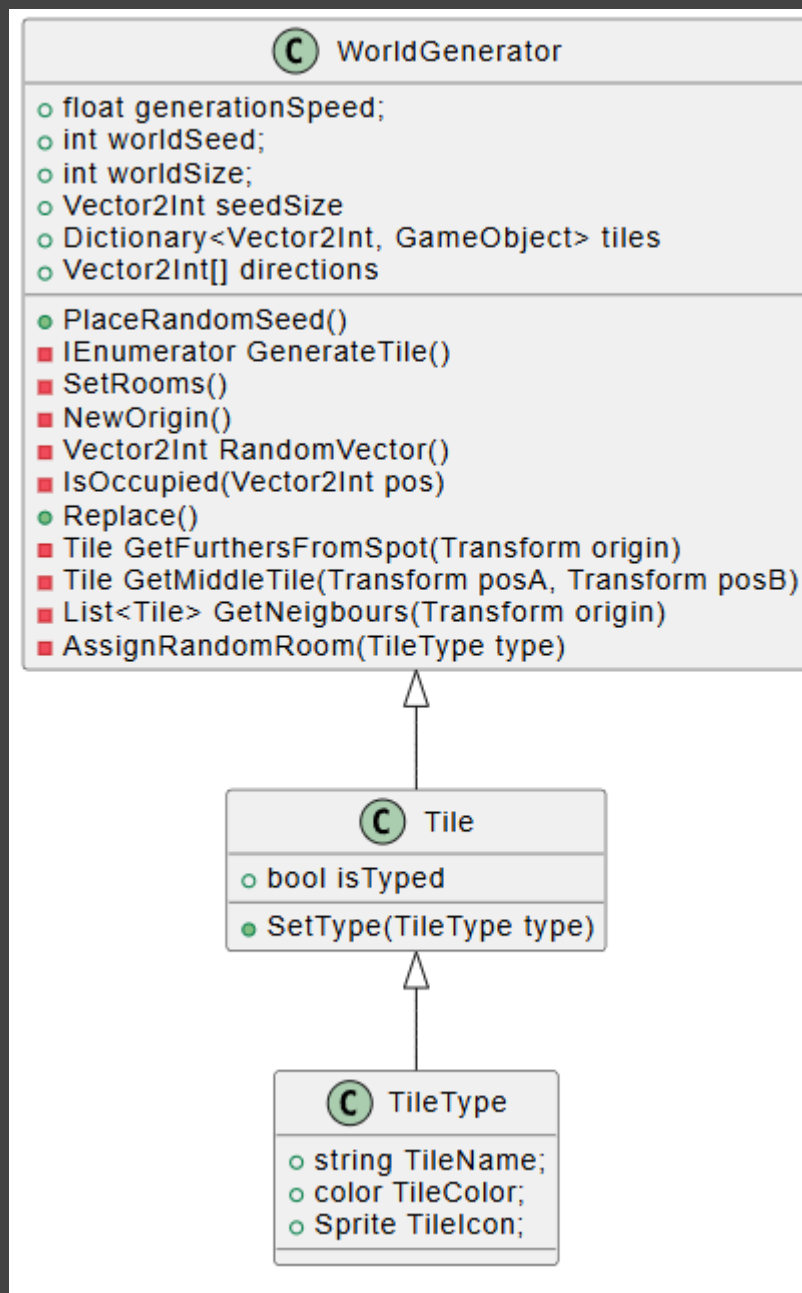
Rule 3: The treasure tile is the tile in between the start and finish

Rule 4: Neighbouring tiles to the treasure room (in cross pattern) will be Danger tiles

Rule 5: any Dead end will be a puzzle tile

Rule 6: A set amount of Combat rooms will be randomly placed throughout the dungeon

UML



PMI

Research	I was able to refer back to used code in different projects and engineer them into this project allowing me to feel more innovative rather than following a guide	It was harder to spend time reading documentation about the theoretical way the dungeon would be generated rather than the sudo code	I did little to in depth research into the actual algorithm Most of my knowledge came from following the classes
Process	It was a lot of fun and I often felt like I wasn't failing my project at every turn which was a nice breath of air because of that I also was able to play around more and make something that also visually give nice feedback	Working with an algorithm I wasn't familiar with and winging it felt surprisingly easy and it is something I believe I should try more to just get projects off the ground	I put way too much time into making it look pretty

Sources

I did not really research aside from interpreting the concepts from class and taking inspirations of grid generation from the **A* assignment**

for the rules I took inspiration from binding of isaac:

https://store.steampowered.com/app/113200/The_Binding_of_Isaac/

But I also mainly took my own initiative for what I would interpret as fun game design within a playable dungeon